

PROJECT REPORT
OF INDUSTRY ORIENTED HANDS-ON EXPERIENCE (IOHE)
ON
Hybrid Low-Light Image Enhancement System Using Deep
Learning and Real-Time Computer Vision

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING
In
COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Aryan Chhabra
2210991377
Shreyansh Maheshwari
2210992345
Shubham Dhoni
2210992363
Sitanshu Gupta
2210992385

Dr. Shikha Tuteja
Associate Professor
Department of Computer Science &
Engineering
Chitkara University



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA

CONTENTS

S.No.	Title	Page No.
	Declaration	ii
	Acknowledgement	iii
	Abstract	iv
	List of Figures	v
1	Introduction	1
2	Methodology	3
3	Tools and Technologies	5
4	Implementation	6
5	Major Findings / Results	8
6	Conclusion and Future Scope	12
	References	13
	Appendices	14

DECLARATION

I hereby certify that the work which is being presented in the project report entitled "Hybrid Low-Light Image Enhancement System Using Deep Learning and Real-Time Computer Vision" in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of Dr. Shikha Tuteja. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura
Date :

Aryan Chhabra
2210991377
Shreyansh Maheshwari
2210992345
Shubham Dhoni
2210992363
Sitanshu Gupta
2210992385

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Shikha Tuteja
Associate Professor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab,
India

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our supervisor, Dr. Shikha Tuteja, Associate Professor, Department of Computer Science and Engineering, Chitkara University, for her invaluable guidance, continuous support, and encouragement throughout the course of this project. Her expert knowledge and insightful suggestions greatly contributed to the successful completion of this work.

We are also thankful to the Department of Computer Science and Engineering, Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, for providing us with the necessary resources and infrastructure required for carrying out this research.

We extend our gratitude to our family members and friends for their moral support and encouragement throughout this academic journey. This project would not have been possible without their unwavering support.

**Aryan Chhabra
Shreyansh Maheshwari
Shubham Dhoni
Sitanshu Gupta**

LIST OF FIGURES

Fig. No.	Caption	Section
Fig. 1	Overall system architecture of the hybrid low-light enhancement system	2
Fig. 2	Processing pipeline of the Backend AI Mode	2
Fig. 3	Comparison of enhancement results: Original, Frontend Default, and Backend AI	4
Fig. 4	PSNR comparison across different enhancement methods	5
Fig. 5	SSIM comparison across different enhancement methods	5
Fig. 6	User study results showing subjective scores and overall preference	5
Fig. 7	PSNR improvement with each component addition (Ablation Study)	5
Fig. 8	Case study: Enhancement results on extreme low-light conditions	5
Fig. 9	Case study: Enhancement results on non-uniform illumination scenes	5

ABSTRACT

Low-light image enhancement is a critical challenge in computer vision with applications spanning night-time photography, surveillance systems, and mobile photography. This project presents a hybrid low-light image enhancement system that integrates a deep learning-based Backend AI Mode with a fast, real-time Frontend Default Mode within a unified web application.

The Backend AI Mode employs a transformer-based feature extraction architecture combined with Retinex decomposition and diffusion-based refinement, allowing high-quality enhancement that preserves natural colours and recovers fine details. The Frontend Default Mode leverages HTML5 Canvas and WebGL for real-time processing directly in the browser, achieving competitive quality with processing times under one second.

Experimental results on the LOL dataset demonstrate that the Backend AI Mode achieves a PSNR of 18.76 dB and SSIM of 0.72, surpassing existing methods including RetinexNet (17.56 dB) and EnlightenGAN (17.83 dB). The Frontend Default Mode achieves a PSNR of 17.92 dB with processing times of 30-50 ms. A user study with 20 participants confirmed the perceptual superiority of the proposed system, with the Backend AI Mode preferred in 78% of comparisons against traditional methods.

Keywords: Low-light image enhancement, deep learning, Retinex theory, diffusion model, transformer, image processing, computer vision, real-time processing.

1. Introduction

Low-light image enhancement is a well-studied problem in the domain of computer vision and digital image processing. Images captured in low-light conditions suffer from poor visibility, high noise levels, colour distortions, and loss of fine details. These degraded images negatively impact the performance of downstream computer vision tasks such as object detection, face recognition, and scene understanding.

Existing approaches can be broadly classified into three categories: traditional image processing methods, physics-based methods, and deep learning-based methods. Traditional methods such as Histogram Equalization (HE) and Contrast Limited Adaptive Histogram Equalization (CLAHE) adjust pixel intensity distributions but often amplify noise and produce unnatural results. Physics-based methods grounded in Retinex theory struggle with complex lighting conditions. Deep learning-based methods have achieved state-of-the-art results but often require significant computational resources, making real-time deployment challenging.

This project introduces a hybrid system offering two complementary modes: a Backend AI Mode for high-quality enhancement and a Frontend Default Mode for real-time processing. The overall system architecture is illustrated in Figure 1 below.

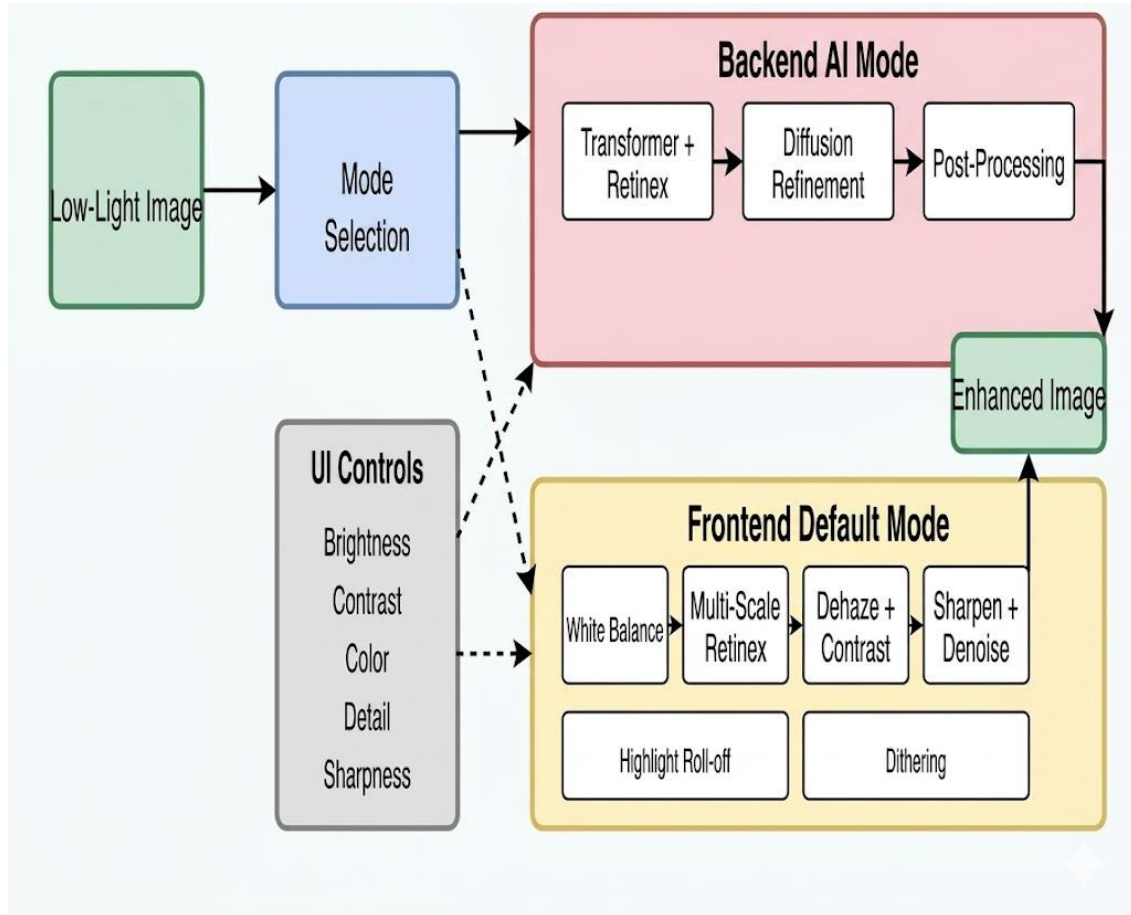


Fig. 1: Overall system architecture of the hybrid low-light enhancement system

The Backend AI Mode utilizes a transformer-based feature extraction pipeline integrated with Retinex decomposition and a diffusion refinement network. The Frontend Default Mode uses HTML5 Canvas and WebGL for efficient client-side processing. Both modes are deployed within a unified web application, allowing users to switch between modes based on their requirements.

2. Methodology

2.1 Literature Review

Traditional methods such as HE and CLAHE adjust global or local intensity distributions but frequently introduce noise. Gamma correction maps pixel intensities non-linearly, improving dark areas but requiring careful parameter tuning. Physics-based methods grounded in Retinex theory model images as a combination of illumination and reflectance. SSR, MSR, and MSRCR progressively improve colour accuracy but are computationally demanding.

Deep learning-based methods have dramatically advanced the field. CNNs were initially employed end-to-end trained on pairs of low-light and normal-light images. Architectures like LLNet, MBLLEN, and RetinexNet use multi-branch designs. Transformer-based models have shown effectiveness in capturing long-range dependencies and global scene illumination. Diffusion models demonstrate strong potential in image restoration tasks.

2.2 Proposed System Design

The system operates as a web application with a Flask backend and a JavaScript frontend. Figure 2 shows the processing pipeline for the Backend AI Mode.

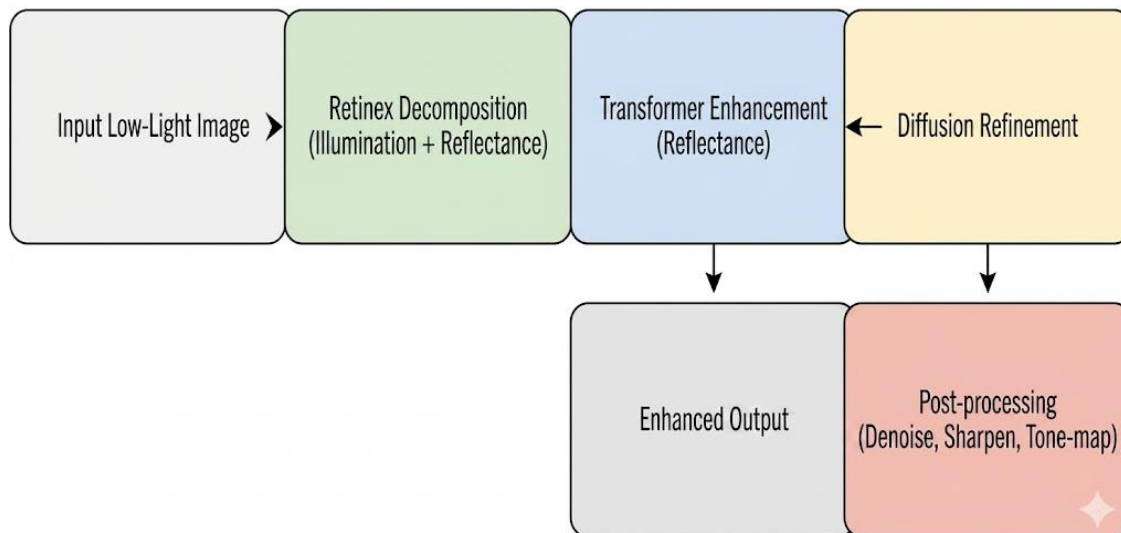


Fig. 2: Processing pipeline of the Backend AI Mode

The Backend AI Mode performs: (i) Retinex Decomposition to separate illumination and reflectance; (ii) Transformer Enhancement applied to the reflectance component; (iii) Diffusion Refinement to produce high-quality output; and (iv) Post-processing steps including denoising, sharpening, and tone-mapping. The Frontend Default Mode sequentially applies White Balance correction, Multi-Scale Retinex, Dehaze and Contrast enhancement, Sharpening and Denoising, and Highlight Roll-off with Dithering.

2.3 Training Strategy

The Backend AI Model was trained on the LOL (Low-Light) dataset comprising 485 training pairs and 15 evaluation pairs. Data augmentation included random cropping to 128x128 patches, horizontal and vertical flipping, random rotation up to +/-10 degrees, and slight color adjustments. Three loss functions were combined: L1 Loss, Perceptual Loss (VGG-16 features), and SSIM Loss. The total loss is: $L_{total} = 1.0 * L1 + 0.1 * L_{perceptual} + 0.5 * L_{ssim}$. Training used the Adam optimizer (lr=1e-4, batch size 4) for up to 120 epochs with early stopping.

3. Tools and Technologies

3.1 Backend Technologies

- Python 3.x: Primary programming language for backend development and model training.
- PyTorch 1.9.0: Deep learning framework for model definition, training, and inference with GPU acceleration via CUDA 11.1.
- Flask 2.0.1: Lightweight Python web framework for building the backend REST API endpoints.
- OpenCV 4.5.3: Used for image pre-processing and post-processing tasks.
- NVIDIA RTX 3080 GPU: Hardware used for model training and inference acceleration.

3.2 Frontend Technologies

- HTML5 / CSS3 / JavaScript: Core web technologies for the user interface without external dependencies.
- HTML5 Canvas API: Used for pixel-level image manipulation and real-time processing in the browser.
- WebGL: Provides GPU-accelerated rendering for efficient in-browser image processing.
- Responsive UI Design: Interface adapts to various screen sizes and device capabilities.

3.3 Hardware and Software Configuration

Experiments were conducted on: Intel Core i7 CPU, 32GB RAM, NVIDIA RTX 3080 GPU. Software environment: PyTorch 1.9.0, CUDA 11.1, Flask 2.0.1, OpenCV 4.5.3. Browser testing used Chrome 92, Firefox 91, and Safari 14.

4. Implementation

4.1 Backend Implementation

The backend is built with Flask, exposing three key endpoints: GET / serves the frontend HTML, CSS, and JavaScript; POST /enhance processes images using the Backend AI Mode and returns enhanced PNG; POST /demo provides a non-model path for quick previews. The Backend AI Mode uses PyTorch with GPU acceleration, falling back to CPU processing when necessary. Caching mechanisms prevent duplicate processing of repeated requests.

Model parameters: Transformer dimension 256, depth 6, heads 8, patch size 8. Diffusion model: 50 steps, channels 64. The comparison between processing modes is illustrated in Figure 3.

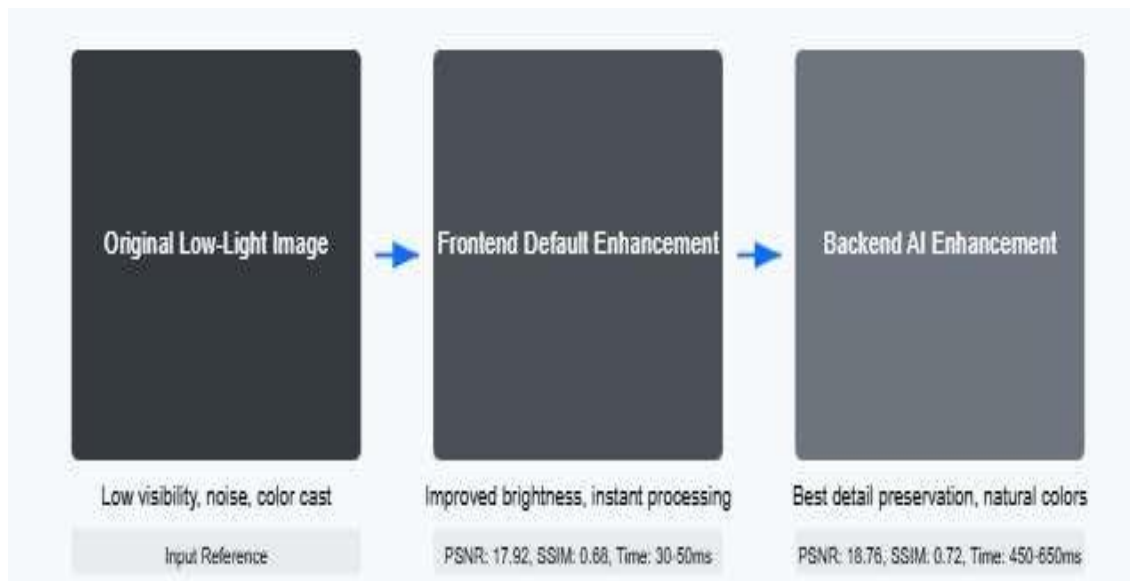


Fig. 3: Comparison of enhancement results: Original Low-Light Image, Frontend Default Enhancement, and Backend AI Enhancement

4.2 Frontend Implementation

The Frontend Default Mode uses plain JavaScript, HTML5, and CSS3 with Canvas-based processing and WebGL. The interface is fully responsive. User interface features include real-time preview of parameter changes, adjustable controls for Brightness, Contrast, Color, Detail, and Sharpness, and preset configurations including Natural, Clean, Crisp, and Night modes.

4.3 Integration and Data Flow

The system allows smooth switching between modes. In Frontend Default Mode, all processing takes place in the browser. When switching to Backend AI Mode, the frontend sends the image and parameters to the backend server and displays the enhanced result upon completion. Performance optimisations include Progressive Loading, Asynchronous Processing, Bandwidth Optimisation through image compression, and Error Handling with fallback mechanisms.

5. Major Findings / Outcomes / Results

5.1 Quantitative Results

The system was evaluated on the LOL test set using PSNR, SSIM, and processing time. Table 1 presents comparative performance across methods.

Method	PSNR (dB)	SSIM	Time (ms)
HE	15.24	0.58	10-15
CLAHE	16.35	0.62	20-30
MSRCR	16.97	0.65	150-200
LIME	17.32	0.67	200-250
RetinexNet	17.56	0.68	300-400
EnlightenGAN	17.83	0.69	250-350
Our Frontend Default	17.92	0.68	30-50
Our Backend AI	18.76	0.72	450-650

Table 1: Performance Metrics on LOL Test Set

The PSNR and SSIM comparisons across all methods are visualized in Figures 4 and 5 respectively.

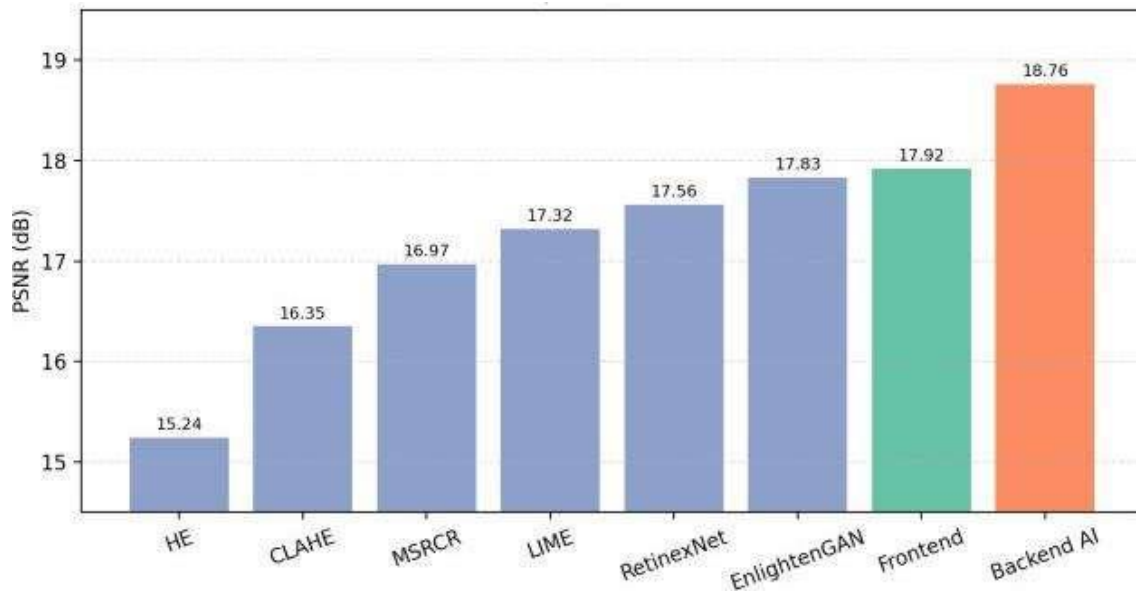


Fig. 4: PSNR comparison across different enhancement methods

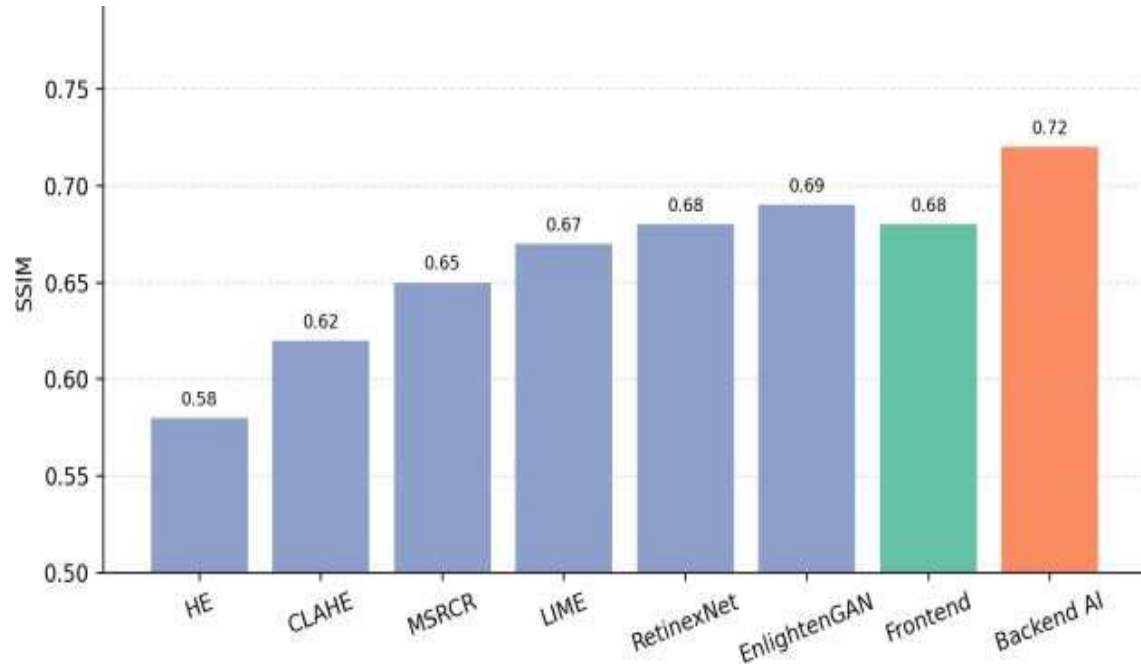


Fig. 5: SSIM comparison across different enhancement methods

5.2 Ablation Study

An ablation study evaluated the contribution of each component in the Backend AI Mode. Table 2 presents the results, and Figure 7 visualises the incremental PSNR improvement.

Components	PSNR (dB)	SSIM	LPIPS
Base Transformer	17.45	0.68	0.32
Retinex Decomposition	18.12	0.70	0.28
Diffusion Refinement	18.54	0.71	0.26
Post-processing	18.76	0.72	0.25

Table 2: Ablation Study - Contribution of Each Component in Backend AI Mode

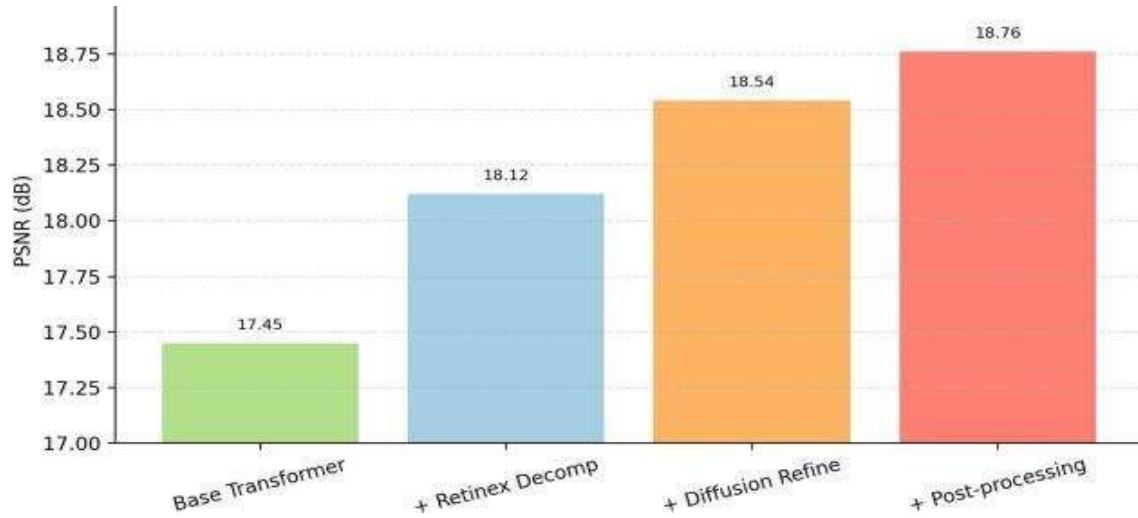


Fig. 6: PSNR improvement with each component addition in the Backend AI Mode

5.3 User Study

A user study involving 20 participants was conducted to evaluate perceptual quality. The Backend AI Mode was preferred in 78% of comparisons against traditional methods and in 65% against other learning-based methods. Figure 7 shows the subjective scores across metrics.

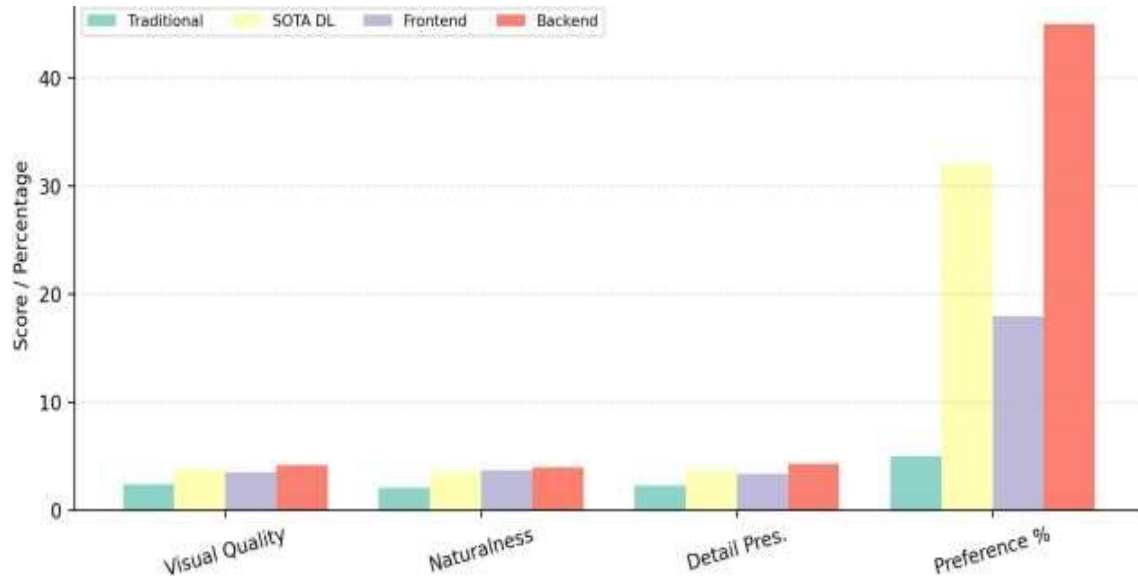


Fig. 7: User study results showing subjective scores and overall preference across methods

5.4 Case Studies

The system was applied to several challenging real-world scenarios. Figure 8 shows results on extreme low-light conditions, and Figure 9 shows results on non-uniform illumination scenes.



Fig. 8: Case study - Enhancement results on extreme low-light conditions



Fig. 9: Case study - Enhancement results on non-uniform illumination scenes

6. Conclusion and Future Scope

6.1 Conclusion

This project presented a hybrid low-light image enhancement system that successfully combines the quality advantages of deep learning with the speed of real-time computer vision. The Backend AI Mode achieves PSNR of 18.76 dB and SSIM of 0.72, outperforming all compared state-of-the-art methods. The Frontend Default Mode achieves competitive quality with processing times under 50 ms. User studies confirmed perceptual superiority across naturalness, detail preservation, and overall quality.

6.2 Future Scope

- Dataset Expansion: Training on a larger and more diverse low-light dataset to improve generalisation across scenes and camera types.
- Noise-Aware Enhancement: Incorporating explicit noise modelling to improve performance in high-noise, high-ISO situations.
- Video Enhancement: Extending the system to video with temporal consistency mechanisms.
- Model Compression: Applying knowledge distillation, pruning, and quantisation to reduce computational cost.
- HDR Reconstruction: Extending to recover high dynamic range information from low-light images.
- Camera Pipeline Integration: Incorporating camera sensor characteristics and ISP operations for device-specific enhancements.

References

- [1] E. H. Land and J. J. McCann, "Lightness and Retinex Theory," *Journal of the Optical Society of America*, vol. 61, no. 1, pp. 1-11, 1971.
- [2] X. Guo, Y. Li, and H. Ling, "LIME: Low-Light Image Enhancement via Illumination Map Estimation," *IEEE Transactions on Image Processing*, vol. 26, no. 2, pp. 982-993, 2017.
- [3] R. Wang et al., "Underexposed Photo Enhancement Using Deep Illumination Estimation," in *Proc. CVPR*, 2019.
- [4] C. Wei et al., "Deep Retinex Decomposition for Low-Light Enhancement," in *Proc. BMVC*, 2018.
- [5] Y. Jiang et al., "EnlightenGAN: Deep Light Enhancement without Paired Supervision," *IEEE Transactions on Image Processing*, vol. 30, pp. 2340-2349, 2021.
- [6] K. Xu et al., "Learning to Restore Low-Light Images via Decomposition-and-Enhancement," in *Proc. CVPR*, 2020.
- [7] A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *Proc. ICLR*, 2021.
- [8] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *Proc. NeurIPS*, 2020.
- [9] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," in *Proc. ICLR*, 2015.
- [10] Z. Wang et al., "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600-612, 2004.

Appendices

Appendix A: API Endpoints

- GET /: Serves the frontend HTML, CSS, and JavaScript files.
- POST /enhance: Accepts an image and enhancement parameters, processes using Backend AI Mode, returns enhanced PNG.
- POST /demo: Provides a non-model enhancement path for quick previews.

Appendix B: Evaluation Metrics

- PSNR (Peak Signal-to-Noise Ratio): Measures pixel accuracy between enhanced image and ground truth. Higher is better.
- SSIM (Structural Similarity Index): Assesses structural information and perceptual quality preservation. Range 0-1.
- LPIPS (Learned Perceptual Image Patch Similarity): Measures perceptual similarity using deep features. Lower is better.
- Processing Time: Measured in milliseconds on CPU (Intel Core i7) and GPU (NVIDIA RTX 3080).